DÉPLOIEMENT D'APPLICATIONS INTELLIGENTES · MLOps

TP de synthèse - Projet « Médicaments » : des données réelles au modèle servi

Synthèse des TP 1, 2 et 3 · 2 séances de 2 h + travail personnel · en binôme · aucun code fourni

Le principe de ce TP

Dans les TP 1, 2 et 3, le code vous était donné : vous l'avez exécuté, observé, puis modifié. Ce TP de synthèse inverse la situation. Aucun code n'est fourni. Vous recevez un cahier des charges, un jeu de données réel publié sur le portail Open Data du Maroc, des points de contrôle chiffrés pour vérifier que vous êtes sur la bonne voie, et la liste de ce qu'il faut rendre. À vous de chercher, d'écrire et de faire fonctionner le projet.

Vos fiches des TP 1 à 3 et le mémento des commandes MLflow restent vos meilleurs points de départ : la démarche est la même, mais les données, les variables et les algorithmes changent, donc aucun fichier ne peut être recopié tel quel.

Phase	Ce que vous construisez	TP repris	Durée conseillée
A	Récupérer les données réelles, les explorer, les transformer en tableau numérique	pré-requis	45 min
B	Comparer au moins quatre algorithmes et tracer chaque essai dans MLflow	TP 1	1 h
C	Pipeline en trois étapes, paramètres externes, règle de validation, `MLproject`	TP 2	1 h 15
D	Signature, registre, alias, retour arrière, service HTTP	TP 3	1 h
E	Compte rendu avec captures des commandes et des résultats	tous	travail personnel

RÈGLE DU JEU

Travail en binôme. Vous pouvez consulter la documentation officielle (MLflow, scikit-learn, pandas), vos TP précédents et le Web. Tout code repris d'une source extérieure doit être compris : à la soutenance, chaque membre du binôme doit savoir expliquer n'importe quelle ligne du projet.

1. Le contexte et la mission

La CNOPS (Caisse Nationale des Organismes de Prévoyance Sociale) publie sur data.gov.ma son référentiel des médicaments : pour chaque présentation commercialisée, le nom, la molécule, le dosage, la forme, les prix et le taux de remboursement. Chaque ligne indique aussi si le médicament est un princeps (le médicament d'origine) ou un générique (une copie autorisée, en général moins chère).

Votre mission : construire un modèle qui prédit, à partir des seules informations de prix, de dosage et de remboursement, si un médicament est un générique (classe 1) ou un princeps (classe 0), puis le livrer comme un vrai produit : suivi des essais, pipeline reproductible, version enregistrée, alias de production, service HTTP.

2. Les données

Élément	Valeur
Portail	https://data.gov.ma (Portail de l'Open Data au Maroc)
Jeu de données	Référentiel des médicaments - producteur : CNOPS
Ressource à télécharger	le fichier XLSX `ref-des-medicaments-cnops-2014.xlsx`
Taille du fichier	530 243 octets
Empreinte MD5	`68a57c72d5eeb4c5256fba3322393d32`
Contenu	5 917 lignes, 12 colonnes, une ligne par présentation de médicament

Pour l'obtenir : ouvrez le portail, cherchez « référentiel des médicaments », ouvrez la fiche du jeu de données, puis téléchargez la ressource au format XLSX. Placez le fichier dans `data/brut/` sans le modifier et vérifiez sa taille et son empreinte. Relevez sur la fiche du portail la licence et la date de mise à jour : vous les citerez dans le compte rendu.

Colonne	Signification	Remarque
CODE	code-barres de la présentation	identifiant, pas une variable
NOM	nom commercial	texte
DCI1	dénomination commune internationale : la molécule	1 134 valeurs différentes
DOSAGE1, UNITE_DOSAGE1	dosage et son unité	valeurs manquantes, parfois non numériques
FORME, PRESENTATION	comprimé, gélule, sirop... et conditionnement	texte
PPV	prix public de vente, en dirhams	peut valoir 0
PH	prix hôpital, en dirhams	peut valoir 0
PRIX_BR	prix servant de base au remboursement	
TAUX_REMBOURSEMENT	taux de remboursement	texte : « 70% » ou « 0% »
PRINCEPS_GENERIQUE	P = princeps, G = générique	LA CIBLE

3. Les contraintes imposées

Elles servent à deux choses : vous obliger à sortir de ce que vous avez déjà fait, et rendre tous les projets comparables à la correction.

- Algorithmes. `RandomForestClassifier` (TP 1 à 3) et LightGBM (TP 3) sont interdits. Vous devez comparer au moins quatre algorithmes appartenant à au moins trois familles différentes parmi : modèle linéaire, plus proches voisins, arbre de décision seul, boosting de gradient, machine à vecteurs de support, bayésien naïf, réseau de neurones simple.
- Variables. Au moins sept variables numériques, dont au moins deux que vous construisez vous-mêmes à partir des colonnes existantes. La cible ne doit évidemment servir à construire aucune variable.
- Découpage. 25 % des lignes pour le test, graine aléatoire 42, pour que vos chiffres soient comparables aux points de contrôle.
- Métrique de décision. Le F1 de la classe « générique », calculé sur le jeu de test. Journalisez aussi la précision et le rappel.
- Règle de validation. Un modèle n'entre au registre que si son F1 est supérieur ou égal à 0,75. Ce seuil est lu dans `params.yaml`, jamais écrit en dur dans le code.
- Noms imposés. Expériences `medicaments-exploration` et `medicaments-pipeline` ; modèle enregistré `medicament-generique` ; alias `production` et `staging`.
- Serveur. Un serveur MLflow local sur le port 5000 avec une base SQLite, comme dans les TP ; le modèle servi sur le port 8000.

ARBORESCENCE ATTENDUE (LES NOMS DES SCRIPTS DE src/ SONT LIBRES)

```
projet-medicaments/
    data/brut/           le fichier XLSX telecharge, jamais modifie
    data/prepare/        train.csv et test.csv, produits par le pipeline
    src/                 un script par responsabilite (exploration, variables, comparaison, 3 etapes)
    models/  reports/    le modele entraine, les metriques et la matrice de confusion
    params.yaml          tous les reglages : decoupage, algorithme, hyperparametres, seuil
    pipeline.py          enchaine les trois etapes et applique la regle de validation
    MLproject            point d'entree standard
    requirements.txt     dependances
```

Phase A - Récupérer, explorer, préparer

Avant tout modèle, il faut connaître ses données. Cette phase ne fait pas encore appel à MLflow.

Ce que vous devez réaliser

1. Créez le dossier du projet, un environnement virtuel et un `requirements.txt`. Indice : pandas ne sait pas lire un fichier XLSX sans une bibliothèque supplémentaire, à vous de trouver laquelle.
2. Écrivez un script d'exploration qui affiche : les dimensions, la liste des colonnes, les valeurs manquantes par colonne, la répartition de la cible, la répartition des taux de remboursement, les statistiques du prix public et le nombre de molécules différentes.
3. Écrivez une fonction unique qui transforme le tableau brut en tableau numérique (variables + cible). Elle sera réutilisée par la comparaison et par le pipeline : le même traitement partout, écrit une seule fois.
4. Décidez et justifiez votre traitement des valeurs manquantes, des dosages non numériques, du taux écrit en texte et des prix nuls (attention aux divisions).

POINTS DE CONTRÔLE - SI VOUS N'OBTENEZ PAS CELA, NE PASSEZ PAS À LA SUITE
Dimensions : 5 917 lignes et 12 colonnes.
Cible : 3 031 princeps et 2 886 génériques (classes presque équilibrées).
Valeurs manquantes : 181 dans `DOSAGE1`, 171 dans `UNITE_DOSAGE1`, aucune ailleurs.
Taux de remboursement : 3 940 lignes à 70 %, 1 977 à 0 %. Prix public : médiane 65,40 DH, maximum 38 480 DH.
1 134 molécules (DCI) différentes.

Captures à placer dans le compte rendu

- la vérification du fichier téléchargé (taille, empreinte) et les versions installées ;
- la commande d'exploration et sa sortie complète.

Phase B - Comparer des algorithmes et tracer les essais (TP 1)

Vous reprenez la démarche du TP 1 : chaque essai laisse une trace complète et comparable. La différence : ce ne sont plus des hyperparamètres d'un même algorithme que vous comparez, mais des familles d'algorithmes.

Ce que vous devez réaliser

1. Démarrez le serveur MLflow dans un terminal dédié.
2. Écrivez un script qui entraîne vos algorithmes (quatre au minimum) sur le même découpage et crée un run par algorithme dans l'expérience `medicaments-exploration`, avec un nom de run lisible.
3. Pour chaque run, journalisez : des paramètres (au moins le nom de l'algorithme et le nombre de variables), les métriques F1, précision, rappel et la durée d'entraînement, et deux tags indiquant le jeu de données et sa source.
4. Certains algorithmes exigent des variables mises à la même échelle, d'autres non. Trouvez lesquels et traitez-les correctement, sans que l'information du jeu de test ne fuie vers l'entraînement.
5. Dans l'interface, comparez les runs (tableau et graphiques) et choisissez votre algorithme. Justifiez le choix par les chiffres, pas par la réputation de l'algorithme.

POINTS DE CONTRÔLE - SI VOUS N'OBTENEZ PAS CELA, NE PASSEZ PAS À LA SUITE
Avec les seules colonnes numériques d'origine et deux variables simples, une régression logistique donne un F1 d'environ 0,63 et les meilleurs algorithmes plafonnent vers 0,72 : le seuil de 0,75 n'est pas atteint.
Il faut donc une variable construite plus riche. Piste : un générique n'existe que parce que d'autres laboratoires commercialisent la même molécule. Quelle information du tableau, calculée par groupe, traduit cette idée ?
Avec une bonne variable de ce type, la référence de l'enseignant obtient entre 0,66 (régression logistique) et 0,81 (meilleur boosting). Des écarts de quelques centièmes avec vos choix sont normaux.

Captures à placer dans le compte rendu

- la commande de lancement du serveur ;
- la sortie du script de comparaison ;

- le tableau des runs dans l'interface MLflow, avec les colonnes de métriques ;
- la vue graphique comparant les runs.

Phase C - Un pipeline reproductible avec règle de validation (TP 2)

Vous transformez l'essai gagnant en pipeline, comme au TP 2 : des étapes séparées, des réglages hors du code, un garde-fou automatique.

Ce que vous devez réaliser

1. Découpez le travail en trois scripts indépendants : préparer (lit le XLSX, écrit `train.csv` et `test.csv`), entraîner (lit `train.csv`, écrit le modèle), évaluer (lit le modèle et `test.csv`, écrit `metrics.json` et une image de la matrice de confusion).
2. Placez tous les réglages dans `params.yaml`. Le choix de l'algorithme doit pouvoir changer sans toucher au code : prévoyez au moins deux algorithmes sélectionnables, votre meilleur et un plus faible.
3. Écrivez `pipeline.py` : un run parent dans `medicaments-pipeline`, un run enfant imbriqué par étape, puis la journalisation des paramètres, des métriques et de la matrice de confusion sur le run parent.
4. Ajoutez la règle de validation. Modèle accepté : il est journalisé et enregistré au registre. Modèle refusé : le run porte un tag qui le dit, rien n'entre au registre, et le programme se termine avec un code de sortie différent de zéro.
5. Provoquez volontairement un refus en sélectionnant l'algorithme faible, constatez-le dans le terminal et dans l'interface, puis revenez au bon algorithme.
6. Écrivez un fichier `MLproject` et relancez le pipeline avec la commande `mlflow run`, sans créer de nouvel environnement.

POINTS DE CONTRÔLE - SI VOUS N'OBTENEZ PAS CELA, NE PASSEZ PAS À LA SUITE
L'étape de préparation annonce 4 437 lignes d'entraînement et 1 480 lignes de test.
Dans l'interface, le run parent se déplie et montre ses trois enfants ; le run refusé apparaît en échec avec son tag.
Après un pipeline accepté, le modèle `medicament-generique` existe au registre en version 1.
`mlflow run` se termine par une ligne contenant « succeeded ».

Captures à placer dans le compte rendu

- le pipeline accepté (terminal) ;
- le pipeline refusé avec son code de sortie (terminal) ;
- la commande `mlflow run` et sa sortie ;
- les runs imbriqués dans l'interface, refus visible ;
- la page d'un run parent : paramètres, métriques, tags ;
- la matrice de confusion affichée dans les artefacts.

Phase D - Signature, registre, alias, service (TP 3)

Le modèle devient un produit versionné que d'autres peuvent appeler, comme au TP 3.

Ce que vous devez réaliser

1. Faites en sorte que le modèle journalisé porte une signature (noms et types des entrées, type de la sortie). Retrouvez-la dans les fichiers du modèle et dans l'interface.
2. Obtenez deux versions au registre : votre meilleur réglage, puis un réglage volontairement plus simple qui passe tout de même le seuil.
3. Écrivez un script qui : liste les versions avec leur F1, place l'alias `production` sur la meilleure et `staging` sur l'autre, ajoute une description à la version de production, charge le modèle par son alias et prédit pour un médicament de votre choix.
4. Dans le même script, simulez une promotion (l'alias `production` passe sur la candidate) puis un retour arrière, en affichant à chaque fois quelle version répond. Le code qui charge le modèle ne doit pas changer d'une ligne.
5. Servez le modèle `production` en HTTP sur le port 8000. Envoyez une requête valide avec deux médicaments réels du fichier (un générique, un princeps), puis une requête volontairement invalide et expliquez la réponse.

POINTS DE CONTRÔLE - SI VOUS N'OBTENEZ PAS CELA, NE PASSEZ PAS À LA SUITE

La signature compte autant d'entrées que vous avez de variables, toutes de type `double`, et une sortie de type `long`.
Le registre montre deux versions, chacune avec son alias.
La requête valide renvoie un JSON de la forme `{"predictions": [1, 0]}`.
La requête invalide est rejetée par le serveur avec un message qui nomme le problème : le modèle n'est jamais appelé avec des données mal formées.

Captures à placer dans le compte rendu

- le contenu du dossier du modèle et la signature ;
- la sortie du script de registre (promotion et retour arrière) ;
- la page du modèle au registre, alias visibles ;
- la requête valide et sa réponse ;
- la requête invalide et le message d'erreur.

Où chercher : des pistes, pas des solutions

Besoin	Mots-clés et pages à consulter
Lire un fichier XLSX	pandas `read_excel`, moteur `openpyxl`
Convertir un texte en nombre malgré des valeurs invalides	pandas `to_numeric`, paramètre `errors`
Calculer une valeur par groupe et la reporter sur chaque ligne	pandas `groupby` puis `transform`
Mettre à l'échelle sans fuite d'information	scikit-learn `Pipeline`, `make_pipeline`, `StandardScaler`
Choisir d'autres algorithmes	guide scikit-learn « Supervised learning » : linear_model, neighbors, tree, ensemble (Gradient Boosting, Hist Gradient Boosting), svm, naive_bayes, neural_network
Matrice de confusion en image	scikit-learn `ConfusionMatrixDisplay`
Runs parent et enfants	MLflow `start_run`, paramètre `nested`
Arrêter un script avec un code d'erreur	Python `SystemExit`, `sys.exit`
Signature et exemple d'entrée	MLflow `log_model`, paramètre `input_example` ; « model signature »
Alias du registre	MLflow `MlflowClient`, « registered model alias » ; adresse `models:/nom@alias`
Servir un modèle et l'appeler	commande `mlflow models serve` ; route `/invocations` ; format `dataframe_split`
Lancer un projet	fichier `MLproject`, commande `mlflow run`, option `--env-manager`

PIÈGES QUE VOUS ALLEZ RENCONTRER (ET QUE VOUS DEVEZ RÉSOUDRE)

Types non approuvés. Depuis MLflow 3.16, les modèles scikit-learn sont enregistrés au format skops. Au chargement, certains algorithmes déclenchent une erreur « untrusted types ». Lisez le message jusqu'au bout : il nomme le paramètre à ajouter et le type à déclarer. Ne contournez pas le problème en changeant de format sans comprendre.

Entiers et décimaux. La signature est stricte : une colonne déclarée `double` refuse un entier. Vérifiez le type de chaque variable avant de journaliser le modèle.

Variable d'environnement. `mlflow run` et `mlflow models serve` ne lisent pas votre code Python : ils doivent connaître l'adresse du serveur autrement.

Sous Windows (PowerShell). `curl` est un alias d'une autre commande et `export` n'existe pas : cherchez `Invoke-RestMethod` et `$env:`.

Phase E - Le compte rendu

Un seul document PDF par binôme, de 10 à 15 pages. Il doit permettre à quelqu'un qui n'a pas assisté au TP de refaire votre projet et de retrouver vos chiffres.

1. Page de garde : noms, groupe, date, système utilisé, versions de Python, MLflow, scikit-learn et pandas.
2. Les données : source, licence, date, contenu, résultats de l'exploration, choix de préparation justifiés.
3. Une section par phase (A à D) : pour chaque étape, la commande utilisée, la capture du résultat obtenu et deux ou trois lignes d'interprétation. Une capture sans commentaire ne rapporte aucun point.
4. Un tableau de synthèse des algorithmes comparés (F1, précision, rappel, durée) et la justification du choix.
5. Les difficultés rencontrées : chaque erreur, son message, sa cause et votre correction.
6. Les réponses aux questions ci-dessous.
7. Annexe : le code complet (ou un lien vers le dépôt).

Captures. Elles doivent être lisibles et montrer la commande et son résultat. Pour le terminal, capturez la fenêtre réelle. Pour l'interface MLflow, cadrez la zone utile. Le nom de votre machine ou de votre session doit rester visible sur au moins une capture par phase.

Questions de réflexion

1. Pourquoi la régression logistique échoue-t-elle ici alors que les méthodes à base d'arbres réussissent ? Appuyez-vous sur la nature des variables.
2. Votre meilleure variable construite utilise-t-elle une information qui serait disponible pour un médicament entièrement nouveau ? Qu'est-ce que cela implique pour le modèle en production ?
3. Le fichier contient des lignes presque identiques (même médicament, codes-barres différents). Quel risque cela fait-il courir à un découpage aléatoire entre entraînement et test ? Comment le mesurer ?
4. Que se serait-il passé, dans une chaîne automatisée, si le pipeline refusé s'était terminé avec le code de sortie 0 ?
5. Lors du retour arrière, qu'est-ce qui a changé côté serveur et qu'est-ce qui n'a pas changé côté application ?
6. Les données datent de 2014. Citez deux raisons concrètes pour lesquelles ce modèle se dégraderait sur le référentiel d'aujourd'hui, et dites comment vous le détecteriez.

Défis bonus (+2 points au maximum)

- Mesurer l'apport d'une variable. Relancez la comparaison avec et sans votre meilleure variable construite et présentez le gain par algorithme.
- Un découpage plus honnête. Refaites l'évaluation en gardant toutes les lignes d'une même molécule du même côté du découpage (cherchez `GroupShuffleSplit`). Commentez l'écart avec votre F1 initial.
- Exploiter le texte. Ajoutez une variable tirée de `FORME` ou de `PRESENTATION` et mesurez son effet.
- Paramètres en ligne de commande. Déclarez des paramètres dans `MLproject` pour choisir l'algorithme avec l'option `-P` de `mlflow run`.

Livrables et barème

- le compte rendu au format PDF ;
- une archive du projet sans l'environnement virtuel, sans `mlflow.db` et sans `mlartifacts/` ;
- le projet doit se relancer avec trois commandes : installer les dépendances, démarrer le serveur, exécuter le pipeline.

Critère	Points
Phase A - données vérifiées, exploration complète, fonction de préparation unique et justifiée	3
Phase B - quatre algorithmes de trois familles, runs complets et comparables, choix justifié par les chiffres	4
Phase C - trois étapes séparées, `params.yaml`, runs imbriqués, refus démontré (tag + code de sortie), `mlflow run`	5
Phase D - signature, deux versions, alias, promotion et retour arrière, requêtes valide et invalide	4
Compte rendu - captures lisibles et commentées, difficultés analysées, réponses aux questions	4
Total	20
Défis bonus	+2

CE QUI FAIT PERDRE DES POINTS
Utiliser RandomForest ou LightGBM. Un seuil écrit en dur dans le code. Des captures sans la commande. Un modèle au registre dont le F1 est sous le seuil. Des chiffres dans le compte rendu qui ne correspondent pas aux captures.